

Argomenti di Teoria LC:

Gerarchia di Chomsky:

1. **Linguaggi di tipo 0:** sono i **linguaggi ricorsivamente enumerabili**, vengono riconosciuti dalle macchine di Turing.
2. **Linguaggi di tipo 1:** sono i **linguaggi dipendenti dal contesto**, vengono riconosciuti dalle macchine di Turing in spazio lineare.
3. **Linguaggi di tipo 2:** sono i **linguaggi liberi dal contesto (CFL)**, vengono riconosciuti dagli automi a pila deterministici.
4. **Linguaggi di tipo 3:** sono i **linguaggi regolari**, vengono riconosciuti dagli automi a stati finiti deterministici e non deterministici (con e senza ϵ -transizioni).

Grammatica:

Una grammatica è una regola tale per cui dato in input un alfabeto restituisce tutte e sole le stringhe appartenenti al linguaggio.

Grammatica context free (CFG):

Una grammatica context free è una quadrupla definita nel seguente modo:

$$G = (V, T, P, S)$$

- V = insieme delle variabili, ogni variabile rappresenta un linguaggio, ossia un insieme di stringhe.
- T = insieme finito di simboli terminali che formano le stringhe del linguaggio da definire.
- P = insieme delle produzioni.
- S = start symbol, rappresenta la definizione ricorsiva di un linguaggio.

Regole:

Le regole sono nella forma:

$$A \rightarrow a$$

- A è un simbolo non terminale.
- a è una stringa di simboli terminali e non terminali

Le regole di produzione possono essere applicate ricorsivamente.

Grammatiche regolari:

Una grammatica regolare è una quadrupla definita nel seguente modo:

$$G = (V, T, P, S)$$

1. ϵ può comparire solo nello start symbol, nel seguente modo $S \rightarrow \epsilon$.
2. Le regole di produzione sono tutte lineari a destra, oppure tutte lineari a sinistra.
3. Le regole di produzione non possono andare da una variabile a due terminali insieme.

NFA (non deterministic finite automata):

Un automa a stati finiti non deterministico è un automa che può trovarsi contemporaneamente in diversi stati, come i DFA gli NFA accettano i linguaggi regolari.

Un NFA è una quintupla:

$$N = (Q, \Sigma, \delta, q_0, F)$$

- Q = l'insieme degli stati.
- Σ = alfabeto dei simboli in input.
- δ = funzione di transizione degli stati.
- q_0 = stato iniziale.
- F = insieme degli stati finali.

La funzione di transizione δ per gli NFA prende in input uno stato e un simbolo restituendo un insieme di zero o più stati.

Base:

Se $|w| = 0$ allora $\delta(q, w) = \hat{\delta}(q, \epsilon) = \{q\}$

Passo:

Se $|w| \geq 1$ allora $w = ax$ con $a \in \Sigma$ e $x \in \Sigma^*$

$\delta(q, a) = \{p_1, p_2, \dots, p_k\}$

$$\hat{\delta}(q, w) = \bigcup_{x=1}^k \hat{\delta}(p_i, x) = \{s_1 \dots s_n\}$$

NFA con ϵ -transizioni:

Gli NFA con ϵ -transizioni, introducono la possibilità di eseguire una transizione senza aver ricevuto in input alla delta alcun simbolo.

Sono definiti come gli NFA ma differiscono nella funzione di transizione δ .

$$N = (Q, \Sigma, \delta, q_0, F)$$

La funzione δ prende in input:

1. Uno stato Q .
2. Un elemento di $\Sigma \cup \{\epsilon\}$ cioè un simbolo di input o ϵ .

$$\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$$

Eclosure(q) = insieme degli stati raggiungibili a partire da q tramite ϵ -transizioni.

È possibile fare l'Eclosure di un insieme di stati, in questo caso restituisce tutti gli stati raggiungibili a partire dai due stati in input.

Definizione per induzione:

- **Base:** $q \in \text{Eclosure}(q)$
- **Passo:** se $p \in \text{Eclosure}(q)$ ed $\exists$ una ϵ -transizione $p \rightarrow r$ allora $r \in \text{Eclosure}(q)$.

Trasformazione da ϵ -NFA a DFA:

1. Calcolare la Eclosure di tutti gli stati presenti nell'automa.
2. Impostare una tabella contenente gli stati sulle righe e i simboli in input sulle colonne. Al primo passaggio viene aggiunta l'Eclosure dello stato iniziale del DFA.
3. Applicare la funzione delta allo stato in modo tale da ricavare per ogni input un insieme di stati. Per rendere più efficiente l'esecuzione aggiungo delle righe alla tabella solo per i nuovi stati.

La funzione delta è la seguente $= \delta_D(S, a) = \text{Eclosure} \left(\bigcup_{p \in S} \delta_N(p, a) \right)$

Trasformazione da ε -NFA a NFA:

Non esiste una trasformazione esplicita da ε -NFA a NFA, tuttavia è possibile trasformare l' ε -NFA in un DFA e successivamente il DFA può essere visto come un caso particolare di NFA. Un DFA è un caso particolare di NFA in cui non esiste alcuna ambiguità nelle transizioni

Trasformazione da ER a ε -NFA:

Si opera per induzione sulla struttura delle espressioni regolari, si costruiscono dei moduli tali che hanno:

- Uno stato accettante.
- Nessun arco entrante nello stato iniziale.
- Nessun arco uscente dallo stato finale.

Base:

1. Se l'espressione regolare è uguale ad ε e quindi $L(R) = \{\epsilon\}$.
2. Se l'espressione regolare è l'insieme vuoto e quindi $L(R) = \emptyset$, lo stato finale non è raggiungibile.
3. Se l'espressione regolare è un singolo simbolo dell'alfabeto e quindi $L(R) = \{a\}$.

Passo induttivo:

1. L'espressione regolare è uguale alla somma di due sotto espressioni, quindi $L(R) = \{L(S) \cup L(T)\}$.
2. L'espressione regolare è uguale alla concatenazione di due sotto espressioni, quindi $L(R) = \{L(S) \circ L(T)\}$.
3. L'espressione regolare è la chiusura di Kleene di una sotto espressione, quindi $L(R) = (L(S))^*$
4. L'espressione regolare è uguale a una sotto espressione di R, quindi $L(R) = L(S)$, il modulo di R è uguale al modulo di S.

DFA (deterministic finite automata):

Un DFA è una quintupla:

$$A = (Q, \Sigma, \delta, q_0, F)$$

- Q = insieme finito e non vuoto di stati.
- Σ = alfabeto dei simboli in input.
- δ = funzione di transizione degli stati.
- q_0 = stato iniziale.
- F = insieme degli stati finali.

La funzione di transizione δ prende in input uno stato e un simbolo dell'alfabeto e restituisce uno stato.

$$\delta: Q \times \Sigma \rightarrow Q$$

Delta cappuccio:

Delta cappuccio è una funzione di transizione, la quale permette di eseguire una mossa di una grammatica.

Minimizzazione del DFA:

Relazione:

Dati due stati p e q questi sono equivalenti se vale $\forall w \in \Sigma^* \delta(p, w) \in F$ se e solo se $\delta(q, w) \in F$.

Basta trovare una stringa per dimostrare che due stati non sono equivalenti.

Se voglio dimostrare che $p \neq q$ basta trovare una stringa $w \in \Sigma^*$ tale che $\hat{\delta}(p, w) \in F$ e $\hat{\delta}(q, w) \notin F$ oppure $\hat{\delta}(p, w) \notin F$ e $\hat{\delta}(q, w) \in F$.

Il processo di minimizzazione del DFA prevede di cercare gli stati equivalenti, in quanto le classi di equivalenza diventeranno gli stati dell'automa minimo. Il primo passo consiste nell'individuare, se presenti, gli stati che non sono raggiungibili, ovvero che hanno solo archi uscenti.

L'algoritmo riempi-tabella funziona nel seguente modo: parte con la stringa vuota come testimone e trova le coppie di stati che sono tra loro distinguibili, esegue un'altra iterazione considerando le stringhe di lunghezza 1, prendendo un simbolo e tutte le coppie di stati e verifica se la computazione porta in una coppia di stati che era già stata scoperta essere distinguibile, in caso positivo la coppia viene marcata come distinguibile. Una coppia di stati risulta distinguibile quando è formata da uno stato finale e da uno non finale. Se durante un'iterazione viene riempita almeno una casella della tabella è necessario effettuare un'altra iterazione. Al termine dell'algoritmo riempi-tabella ci saranno delle celle non marcate, queste celle rappresentano gli stati che non sono distinguibili, tali stati sono equivalenti e possono quindi essere uniti in un unico stato.

L'automa minimo ha come stati le classi di equivalenza, la classe di equivalenza che contiene lo stato iniziale del DFA sarà lo stato iniziale dell'automa minimo, così come quella che contiene lo stato finale sarà lo stato finale dell'automa minimo.

Trasformazione da DFA a ER:

Una espressione regolare è una sequenza di simboli che identifica un insieme di stringhe. Le espressioni regolari possono definire tutti e soli linguaggi regolari e sono formate dai seguenti simboli:

- Operatore $+$: indica la scelta tra due elementi
- Operatore concatenazione : indica entrambi gli elementi.
- Operatore $*$: indica che l'elemento può essere scelto un numero arbitrario di volte, anche 0.

La precedenza tra gli operatori è : $*$ - concatenazione - $+$.

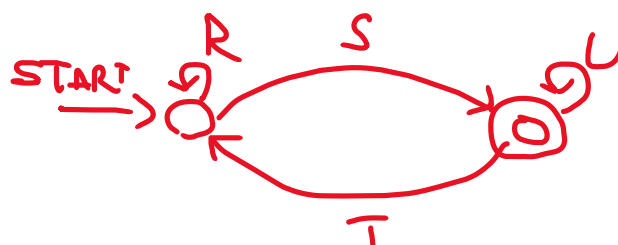
Algoritmo per eliminazione degli stati consiste nei seguenti passaggi:

- Trasformare di simboli degli archi in espressioni regolari.
- Eliminare tutti gli stati eccetto quello iniziale e quello finale.
- Scrivere le espressioni regolari, ogni espressione denota tutte le stringhe che partono dallo stato iniziale e terminano la loro computazione nello stato finale.

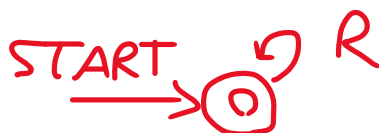
L'espressione regolare che denota il linguaggio accettato dall'automa è la somma di tutte le sotto-espressioni.

Dopo aver eseguito la procedura posso trovarmi in uno dei seguenti casi:

1. Nel caso più generale ci sono uno stato iniziale e uno finale tra loro collegati, un arco se non è presente è come se ci fosse ma etichettato dall'insieme vuoto. L'espressione regolare che denota il linguaggio accettato dal seguente automa è : $(R + SU^*T)^*SU^*$



2. Un altro caso possibile prevede che lo stato iniziale sia anche quello finale, l'espressione regolare che denota il linguaggio accettato in questo caso corrisponde a : R^*



PDA (push-down automata):

Un automa a pila è a un automa a stati finiti non deterministico con ϵ -transizioni e uno stack in cui può memorizzare una stringa di simboli. Gli automi a pila riconoscono i linguaggi liberi dal contesto.

Un PDA è una settupla:

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

- Q = insieme finito degli stati.
- Σ = alfabeto dei simboli in ingresso.
- Γ = alfabeto dei simboli della pila.
- δ = funzione di transizione degli stati.
- q_0 = stato iniziale.
- Z_0 = indica la fine della pila.
- F = insieme degli stati finali.

La funzione di transizione δ prende in input uno stato, un simbolo dell'alfabeto e un simbolo della pila e restituisce un insieme di coppie in cui il primo elemento rappresenta lo stato e il secondo elemento è una stringa di simboli.

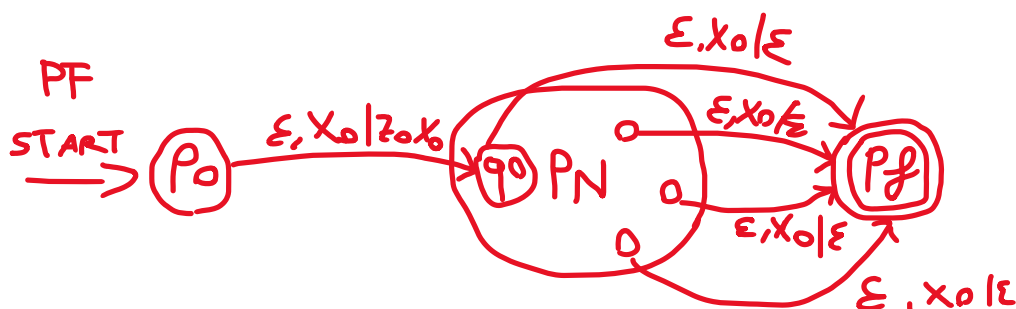
$$\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$$

Trasformazione da pila vuota a stato finale:

Costruisco l'automata P_F che è più grande dell'automata P_N , ha uno stato finale P_f . P_N è un DFA che accetta per pila vuota, in modo tale da poter accettare per pila vuota elimina tutti gli elementi dalla pila compreso Z_0 . Essendo la pila vuota non può accettare per stato finale perché non esiste un caso della delta che funziona con l'elemento in cima alla pila nullo. È necessario quindi aggiungere alla pila un nuovo simbolo iniziale X_0 sconosciuto a P_N ponendolo sotto Z_0 .

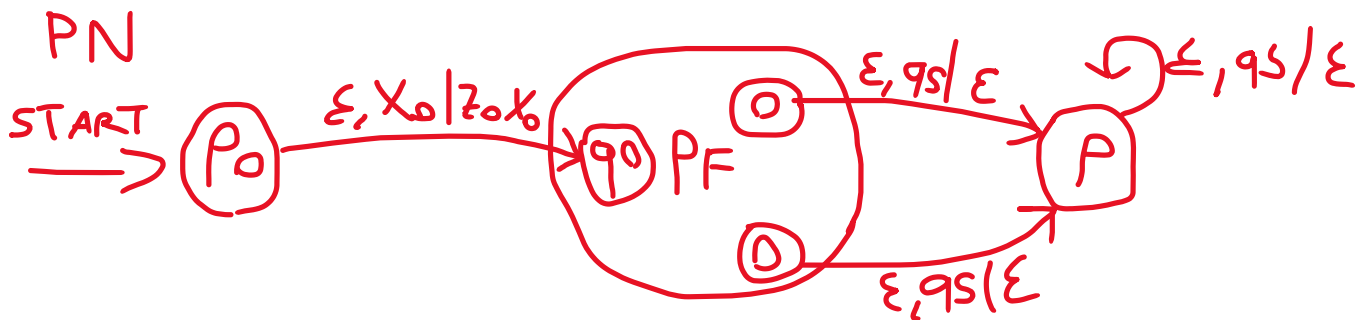
P_F è il nuovo automa che accetta per stato finale, ha uno stato iniziale P_0 , esegue una transizione che impila X_0 sotto Z_0 e passa il controllo a P_N che è libero di consumare tutti i simboli nella pila eccetto X_0 perché non fa parte del suo alfabeto. P_F può riprendere il controllo e se in cima alla pila c'è X_0 con una transizione può raggiungere lo stato finale e accettare.

$$P_F = (Q \cup \{P_0, P_f\}, \Sigma, \Gamma \cup \{X_0\}, \delta_F, P_0, X_0, \{P_f\})$$



Trasformazione da stato finale a pila vuota:

Costruisco l'automa P_N che simula P_F . L'automa P_F parte dallo stato iniziale q_0 , consuma la stringa in input e se si ferma in uno stato finale accetta. Accettando per stati finali la pila può contenere qualsiasi simbolo; quindi, per eseguire la trasformazione è necessario aggiungere un nuovo stato P in cui viene svuotata la pila. L'automa P_F può raggiungere lo stato P solamente a partire dagli stati finali. Questa soluzione presenta un problema, ovvero P_F consumando la pila potrebbe cancellare anche z_0 , così facendo non potrebbe accettare per stati finali in quanto la pila risulterebbe vuota, al contrario P_N potrebbe accettare per pila vuota. Per ovviare a questo problema si aggiunge uno stato iniziale P_0 in cui non si consuma nulla in input e si aggiunge il simbolo x_0 sotto z_0 all'interno della pila. Così facendo anche nel caso in cui la pila venisse svuotata da P_F rimarrebbe il simbolo x_0 in fondo alla pila che non è riconosciuto da P_F perché non è nel suo alfabeto e P_N sarebbe in grado di accettare.



$$P_N = (Q \cup \{P_0, P\}, \Sigma, \Gamma \cup \{X_0\}, \delta_N, P_0, X_0)$$

Descrizione istantanea di un PDA:

Un PDA passa da una configurazione ad un'altra reagendo ai simboli di input, tale configurazione non comprende solo uno stato ma bensì anche il contenuto dello stack. Pertanto, la configurazione può essere mostrata come una tripla (q, w, γ) :

- q è lo stato.
- w è l'input residuo nella stringa.
- γ è il contenuto dello stack.

DPDA (deterministic push-down automata):

Un PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ è deterministico se vengono soddisfatte le seguenti condizioni:

- Uno stato non può avere due archi uscenti con lo stesso simbolo.
- Se uno stato ha un arco uscente con ϵ non può avere nessun altro arco.

Differenze tra PDA e NPDA:

Esistono due tipi di automi a pila: NPDA, automi a pila non deterministici e DPDA, automi a pila deterministici. Gli automi a pila non deterministici NPDA che accettano per stati finali e per pila vuota riconoscono tutti i linguaggi liberi dal contesto CFL, un DPDA invece può accettare solo una sottoclasse di questi, i linguaggi regolari e alcuni CFL. Ad esempio, il linguaggio delle stringhe palindrome pari, costituito dalle stringhe $L = ww^r$ non può essere accettato in modo deterministico perché sarebbe necessario indovinare la metà dell'input. Un DPDA accetta solo CFG non ambigue (una grammatica è ambigua quando ha più di un albero sintattico), infatti nella trasformazione da PDA a CFG equivalente ogni derivazione a sinistra corrisponderà a una computazione del DPDA, poiché un DPDA deterministico ha solo una computazione, ci sarà una derivazione a sinistra unica. I NPDA accettano sia per stato finale che per pila vuota indipendentemente dal fatto che il linguaggio sia prefix-free o meno, mentre per quanto riguarda i DPDA, Se un linguaggio gode della PP può essere

riconosciuto sia per stato finale che per pila vuota, se non gode della PP può accettare solo per stato finale.

È possibile dimostrare che un PDA accetta lo stesso linguaggio per sia per stato finale che per pila vuota in quanto è possibile effettuare la trasformazione da pila vuota a stato finale e viceversa.

Perchè i DPDA che accettano per pila vuota possono accettare solo i linguaggi che sono prefix-free?

Un automa a pila deterministico DPDA che accetta per pila vuota può riconoscere solo linguaggi che sono prefix-free perché, se il linguaggio non fosse prefix-free, ci potrebbero essere stringhe che sono prefisse di altre stringhe del linguaggio, in questo caso il DPDA svuoterebbe la pila dopo aver letto il prefisso e non sarebbe in grado di accettare la stringa più lunga.

Dimostrazione del fatto che i PDA e i DPDA riconoscono almeno tutti i linguaggi regolari:

I linguaggi regolari sono riconosciuti sia dagli automi a stati finiti deterministici DFA che non deterministici NFA. Poiché sia i DFA che gli NFA possono essere simulati da un automa a pila PDA o un automa a pila deterministico DPDA, segue che i PDA e i DPDA possono riconoscere almeno tutti i linguaggi regolari.

Un PDA può simulare un DFA o un NFA mantenendo lo stato corrente del DFA/NFA nel suo stato e leggendo l'input senza mai utilizzare la pila.

Un DPDA può simulare un DFA in modo simile a un PDA; tuttavia, non può simulare un NFA in generale, poiché un NFA può richiedere comportamenti non deterministici che un DPDA non può fornire. Dal momento che ogni linguaggio regolare può essere riconosciuto da un DFA, segue che un DPDA può riconoscere almeno tutti i linguaggi regolari.

Trasformazione da CFG a PDA:

Per costruire un automa che accetta una grammatica libera dal contesto (CFG), è possibile utilizzare un PDA ed effettuare i seguenti passaggi:

- Per ogni produzione nella CFG si aggiunge una transizione nell'automa a pila.
- Ogni variabile viene consumata secondo le regole di produzione.
- L'automa inizia con il simbolo di inizio della grammatica in cima alla pila.
- L'automa accetta una stringa se riesce a consumare l'intera stringa e svuotare la pila.

Estensioni delle macchine di Turing:

Macchina di Turing in cui la testina può rimanere ferma durante la computazione:

$\delta Q \times \Gamma \rightarrow Q \times \Gamma \times (L, S, R)$ in questa versione estesa della macchina di Turing la testina oltre che muoversi a destra e a sinistra lungo il nastro, può rimanere ferma. Simulando questa macchina con due passi di quella di base è possibile mostrare che sono equivalenti.

Suppongo di avere due modelli di calcolo differenti una MT M_1 e una macchina a registri M_2 . Se ogni computazione della macchina M_1 può essere simulata con la macchina M_2 significa che le macchine della classe M_1 non possono essere più potenti di quelle della classe M_2 .

Se faccio vedere la simulazione inversa, dove ogni macchina M_2 la posso simulare con una macchina M_1 arrivo a dimostrare che le macchine hanno la stessa capacità computazionale.

Le macchine tradizionali che non hanno la stay, possono simulare questa estensione, analogamente le macchine che hanno la possibilità di muovere la testina possono simulare quelle tradizionali non utilizzando la stay.

Macchina di Turing multinastro (con un numero finito di nastri):

Nella configurazione iniziale il controllo finito si trova nello stato q_0 , il primo nastro contiene la stringa w con la testina posizionata sul primo nastro. La funzione di transizione δ prende in input lo stato q , i simboli che si trovano sotto tutte le testine degli altri k -nastri e produce in output un simbolo e la direzione di spostamento della testina.

Un passo di computazione è il seguente:

$$\delta(q, X_1, X_2, \dots, X_k) \rightarrow (P, Y_1, Y_2, Y_k, \dots, D_1, D_2, \dots, D_k).$$

Per simulare le macchine di Turing tradizionali, la macchina di Turing con un singolo nastro avrà come alfabeto le k -uple degli alfabeti dei simboli di nastro della macchina con k -nastri, con un simbolo riesce a rappresentare il simbolo sotto ogni testina. Le macchine di Turing con singolo nastro possono simulare quelle con k -nastri quindi le macchine multinastro hanno la stessa capacità computazionale di quelle mono nastro e riconoscono tutti i linguaggi ricorsivamente enumerabili.

Macchine di Turing non deterministiche:

Nelle macchine di Turing non deterministiche la funzione δ non ha più una tripla di output, ma un insieme di triple di output:

$$\delta Q \times \Gamma \rightarrow 2^{Q \times \Gamma \times \{L, R\}}$$

Nella computazione di una macchina di Turing non deterministica c'è un albero di computazione associato. L'albero avrà come nodi le configurazioni, come radice dell'albero la configurazione iniziale, alcune configurazioni saranno delle foglie, ovvero la δ non sarà definita su quella configurazione. Se la macchina in quella configurazione si trova in uno stato finale, la configurazione sarà accettabile altrimenti non accettabile.

La macchina è accettabile se esiste almeno una computazione che porta ad accettare, per scoprire le configurazioni accettabili si può duplicare/triplicare la macchina e se una configurazione si ferma in una configurazione accettabile la stringa è accettata.

Le macchine di Turing deterministiche e non deterministiche sono equivalenti in termini di capacità computazionale; infatti, entrambe riescono a riconoscere i linguaggi ricorsivamente enumerabili.

Una macchina di Turing deterministica è fatta nel seguente modo:

$$q_0 w \rightarrow c_1 \rightarrow c_2$$

È un caso particolare della macchina di Turing non deterministica, infatti rappresenta un solo cammino dell'albero. Quindi è possibile affermare che le macchine di Turing non deterministiche sono potenti tanto quanto quelle deterministiche.

Per dimostrare che le macchine di Turing non deterministiche possono simulare quelle deterministiche è possibile utilizzare la macchina multistrato.

$q_0 w$				
---------	--	--	--	--

c_1	c_2	c_3	c_4	c_5	Coda
-------	-------	-------	-------	-------	------

A partire dall'input calcola quali sono le possibili configurazioni e le scrive sul secondo nastro. Elimina l'input iniziale e lo sostituisce con la prima configurazione calcolata, cancellando la configurazione dalla coda. Ripete lo stesso procedimento per le altre configurazioni, così facendo esplora l'albero di computazione per livelli, quindi in ampiezza. Ciò assicura che nel caso ci fosse una configurazione accettabile in qualche livello, si ferma accettando.

Restrizioni delle macchine di Turing:

Ogni linguaggio accettato da una macchina M_2 è accettato anche da una macchina M_1 tale che valgono i seguenti vincoli:

1. La testina non va mai a sinistra della posizione iniziale.
2. M_1 non scrive mai blank.

La restrizione delle macchine di Turing può essere utile per avere dei modelli di calcolo semplificati ma che hanno la stessa potenza computazionale di quelli non semplificati.

Per simulare una macchina di Turing normale, utilizza un nastro “doppio” che è formato dal nastro a sinistra della posizione iniziale ribaltato e unito a quello a destra della testina:

x_0	a	x_2	x_3	...
*	b	$x-2$	$x-3$	...

1. Viene definito un nuovo alfabeto per il nastro, in cui le coppie corrispondono ad una sola cella, ovvero ad un unico simbolo.
2. Viene definito un nuovo simbolo di blank B' ogni volta che dovrei scrivere B scrivo B' .

La coppia $(a, b) = \alpha$ corrisponde al simbolo a del nuovo alfabeto, se volessi scrivere c , al posto della b nella cella (a, b) , la coppia diventerebbe $(a, c) = \beta$ e verrebbe associata ad un nuovo simbolo dell'alfabeto.

Le macchine di turing con un nastro semi-infinito riescono ad accettare tutti i linguaggi ricorsivamente enumerabili perché riescono a simulare le macchine di turing tradizionali.

Macchine di Turing Multistack:

Una macchina di turing multistack è un PDA deterministico con K -pile. Una macchina multistack ha un controllo finito, cioè uno stato preso da un insieme finito; ha un alfabeto di stack finito che è lo stesso per tutti gli stack.

In una mossa la macchina compie diverse operazioni.

- a. Cambia stato.
- b. Sostituisce i simboli in cima agli stack con stringhe di zero o più simboli.

La regola di transizione per una macchina con k stack ha quindi questa forma:

$$\delta(q, a, X_1, X_2, \dots, X_k) = (p, \gamma_1, \gamma_2, \dots, \gamma_k)$$

Nello stato q con X_i in cima all' i -esimo stack, la macchina può consumare a (un simbolo in input oppure ϵ) dall'input, passare allo stato p e sostituire X_i in cima all' i -esimo stack con la stringa γ_i . Le macchine multistack accettano entrando in uno stato finale.

Si sfrutta il fatto che due pile possono simulare il nastro di una macchina di Turing; uno stack conserva ciò che sta a sinistra della testina, l'altro ciò che sta a destra. La macchina S copia l'input w sul primo stack, travasa il contenuto del primo stack all'interno del secondo, ora il primo stack è vuoto, mentre il secondo contiene w , con l'estremo sinistro in cima.

La macchina S simula un passo della macchina M , memorizza lo stato della macchina M all'interno del suo stato, S conosce il simbolo X che la testina di M sta guardando perché è quello in cima al suo secondo stack; quindi, S conosce la mossa di M . Se la macchina M esegue una computazione a destra sostituendo un simbolo, la macchina S sostituisce il simbolo in cima alla pila con quello nuovo e lo

sposta nello stack di destra. Al contrario se la macchina M esegue una computazione a sinistra S toglie il simbolo in cima al primo stack (per esempio Z) e sostituisce X con ZY nel secondo stack. S accetta se il nuovo stato di M è accettante, altrimenti simula un'altra mossa di M nello stesso modo. Con una singola pila non riesce a riconoscere tutti i linguaggi contex-free ma con due pile riesce a riconoscere anche i linguaggi ricorsivamente enumerabili.

Codifica binaria della macchina di Turing:

Le macchine di turing si presentano come una settupla $M = \{Q, \{0, 1\}, \Gamma, \delta, q_1, B, F\}$, in cui gli stati sono $q_0, q_1, \dots, q_r$ per un certo r . Lo stato iniziale sarà sempre q_1 e q_2 sarà l'unico stato accettante, i simboli del nastro sono $X_1, X_2, X_3, \dots, X_s$ per un certo s , X_1 sarà sempre il simbolo 0, X_2 sarà sempre il simbolo 1, X_3 sarà il simbolo blank. Gli altri simboli di nastro possono essere assegnati arbitrariamente agli altri interi. Le direzioni in cui si può muovere la testina sono indicate dal simbolo $m = 1$ per la mossa L e $m = 2$ per la mossa R .

Si codifica ogni caso della delta secondo la regola $(q_i, X_j) = (q_k, X_l, D_m)$ per certi valori interi i, j, k, l, m . Questa regola viene codificata per mezzo della stringa $0^i 10^j 10^k 10^l 10^m$. Poiché i, j, k, l, m valgono ciascuno almeno 1, nel codice di una transizione non compaiono mai due o più 1 consecutivi.

Più codifiche della delta vengono separate da una coppia di 1: $C_1 11 C_2 11 C_3 11 \dots C_{n-1} 11 C_n$

Utilizzi:

La codifica della macchina di Turing in binario in generale serve a dare la codifica di una macchina in input alla macchina di turing più in particolare viene utilizzata nella macchina di turing universale e anche per dimostrare che esiste almeno un linguaggio non ricorsivamente enumerabile come il linguaggio di diagonalizzazione.

Linguaggio di diagonalizzazione:

Il linguaggio di diagonalizzazione L_d è l'insieme delle stringhe w_i tali che w_i non è in $L(M_i)$, la macchina M_i che è codificata dalla stringa w_i prende in input la stringa w_i e la rifiuta.

I linguaggi ricorsivamente enumerabili sono riconosciuti dalle macchine di Turing. Le TM sono denotate da stringhe binarie, però non tutte le stringhe binarie corrispondono a delle TM (ad esempio una stringa che incomincia per "1).

Dimostrazione che il linguaggio di diagonalizzazione non è ricorsivamente enumerabile.

Per dimostrare che il linguaggio di diagonalizzazione non è ricorsivamente enumerabile è necessario costruire una tabella infinita in cui le colonne sono associate all'enumerazione delle stringhe binarie e le righe sono associate all'enumerazione delle macchine di Turing.

Nella tabella alla posizione riga i colonna j abbiamo un 1 se la stringa w_j appartiene al linguaggio accettato da M_i e 0 altrimenti.

L' i -esima riga della matrice rappresenta il vettore caratteristico del linguaggio $L(M_i)$, gli 1 in questa riga indicano le stringhe che appartengono al linguaggio. I valori sulla diagonale segnalano se M_i accetta w_i

Per costruire $L_d = \{w_i \in \{0,1\}^* \mid w_i \notin L(M_i)\}$ si complementa la diagonale e si mostra che corrisponde al vettore caratteristico del linguaggio di diagonalizzazione. Il vettore caratteristico si trova in disaccordo con ogni riga della tabella in almeno una colonna; quindi, il complemento della diagonale non può essere il vettore caratteristico di una macchina di Turing.

Linguaggi ricorsivi:

Un linguaggio ricorsivo è un linguaggio per il quale esiste una macchina di Turing che, data una qualsiasi stringa di input, termina accettando la stringa se essa appartiene al linguaggio e termina ugualmente rifiutando la stringa in caso contrario.

Linguaggi ricorsivamente enumerabili:

Un linguaggio è RE se esiste una MT che accetta il suo linguaggio $L(M)$. Ossia, data una qualunque stringa del linguaggio in input, riesce a determinare che la stringa appartiene al linguaggio in un numero finito di mosse.

Complementi dei linguaggi ricorsivi:

Un modo efficace per dimostrare che un linguaggio è ricorsivamente enumerabile ma non ricorsivo è lo studio del suo complemento. È possibile dimostrare che i linguaggi ricorsivi sono chiusi rispetto all'operazione di complementazione. Perciò se un linguaggio L è RE, ma il suo complemento $\bar{L}$ non lo è, L non può essere ricorsivo. Infatti, se L fosse ricorsivo, anche $\bar{L}$ sarebbe ricorsivo e dunque RE.

Esempio:

Un esempio di linguaggio ricorsivamente enumerabile ma non ricorsivo è il linguaggio universale Lu . Il linguaggio universale è l'insieme delle stringhe binarie che codificano coppie (M, w) , dove M è una MT con alfabeto binario in input e w è un input da essa accettato.

Dimostrazione:

Supponiamo che Lu sia ricorsivo allora anche il complemento di Lu è ricorsivo. Supponendo di avere macchina di turing che accetta il complemento di Lu questa macchina dovrebbe essere in grado di accettare anche il linguaggio Ld . Sappiamo che Ld non è accettato da nessuna macchina di turing quindi non esiste un macchina di turing che accetta il complemento del linguaggio Lu . Il complemento di Lu risulta non ricorsivamente enumerabile e quindi Lu non può essere ricorsivo.

Macchina di turing:

La macchina consiste di:

- Un controllo finito che può trovarsi in uno tra un insieme di stati possibili.
- Un nastro, diviso in celle che possono contenere simboli, di lunghezza infinita.
- Un input, cioè una stringa di lunghezza finita di simboli scelti da un alfabeto di input posizionato sul nastro.
- una testina, un elemento che punta sempre ad una sola cella del nastro, serve per leggere/scrivere nella cella.

La macchina di turing può essere vista come una settupla: $M = \{Q, \Sigma, \Gamma, \delta, q_0, B, F\}$

- Q = insieme degli stati del controllo finito.
- Σ = insieme di simboli di input.
- Γ = insieme dei simboli del nastro (sottoinsieme dei simboli di input)
- δ = funzione di transizione.
- q_0 = stato iniziale.
- B = simbolo blank.
- F = insieme degli stati finali.

La funzione di transizione δ della macchina di turing è così composta:

$$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

Prende in input lo stato corrente e il simbolo puntato dalla testina, esegue una computazione e restituisce uno stato, un nuovo simbolo e la direzione di spostamento della testina, left o right.

La striga non viene consumata ma viene letta avanti e indietro più volte.

Macchina di turing universale:

La macchina di turing universale è una macchina di turing in grado di simulare altre macchine di turing. Riceve in input la descrizione di un'altra macchina di turing T e un possibile input x di T , esegue la computazione $U(T, x)$ il cui esito coincide con quello della computazione $T(x)$.

La macchina di turing universale utilizza generalmente 4 nastri:

- Il primo nastro contiene la descrizione della macchina T che si vuole simulare e la stringa di input
- Il secondo nastro contiene il nastro simulato di T , codificato secondo la codifica binaria delle macchine di turing.
- Il terzo nastro contiene lo stato attuale della macchina di turing presa come input.
- Il quarto nastro funge da nastro ausiliario.

Linguaggio universale:

Lu è l'insieme delle stringhe binarie che codificano coppie (M, w) , dove M è una MT con alfabeto binario in input e w è un input da essa accettato.

Teorema di Rice:

Ogni proprietà non banale dei linguaggi RE è indecidibile.

Complementi dei linguaggi:

Il complemento di un linguaggio ricorsivo è anch'esso ricorsivo, mentre il complemento di un linguaggio ricorsivamente enumerabile è non ricorsivamente enumerabile.